

PCEP SURVIVAL WALL

ANDY'S EDITION V2.0

1. PYTHON OBJECT HIERARCHY

VALUE TYPES

- IMMUTABLE (Cannot change value in-place)**
 - int → 42, -5, 0o12 (Octal), 0x2A (Hex)
 - float → 3.14, -0.0, 2e3 (= 2000.0)
 - bool → True, False [True = 1, False = 0]
 - str → "Andy", 'PCEP', `\"\\\"Multi-line\\\"\"`
 - tuple → (1, 2), (5,) [🚫 Needs trailing comma!]
- MUTABLE (Can change value in-place)**
 - list → [1, 2, 3], ['x', 'y']
 - dict → {"key": "value"}, {1: "one"}
 - set → {1, 2, 3}
- SPECIAL**
 - None → None (Absence of value)

🚫 EXAM DATA TYPE DISGUISES

Literals containing a dot or exponent are always **floats** (e.g. `1.` or `2e0` are floats, not ints). Modifying an immutable tuple item directly via index (`t[0] = 1`) causes a **TypeError**.

2. VARIABLE / VALUE ASSIGNMENT DECODER

```
x = 42
```

TOKEN	PCEP TERM	EXAM ROLE
x	Variable	Name assigned to a value object.
=	Assignment	Binds name on the left to value on the right.
42	Value	The concrete data payload held in memory.

💡 VARIABLE ≠ VALUE

Changing a variable (`x = 20`) does not change the initial value object (`42`); it just reassigns the name label to point somewhere else.

3. FUNCTION ANATOMY DECODER

[DEFINITION SIGNATURE]

```
def add ( a , b ) :
```

- KEYWORD** (Starts block definition)
- FUNCTION NAME (Identifier)**
- PARAMETERS** (Placeholders inside signature)

return a + b

- KEYWORD** (Exits function, sends value back)
- EXPRESSION** (Evaluates to a value)

[CALL & ASSIGNMENT]

```
result = add ( 2 , 3 )
```

- VARIABLE** (Receives return value)
- ASSIGNMENT OPERATOR**
- FUNCTION CALL**
- ARGUMENTS** (Actual values passed to call)

🔄 VISUAL CALL FLOW TRACE

- Code hits: `result = add(2, 3)`
- Arguments map to parameters → `a=2, b=3`.
- Expression runs: `2 + 3` evaluates to `5`.
- `return 5` stops function execution and sends `5` back.
- Variable `result` is assigned the integer value `5`.

4. PARAMETER VS ARGUMENT

DEFINITION BLOCK

```
def greet(name):
```

↓

[PARAMETER]
Placeholder

EXECUTION CALL

```
greet("Andy")
```

↓

[ARGUMENT]
Actual Value

💡 MNEMONIC:

PARAMETER = PLACEHOLDER • ARGUMENT = ACTUAL VALUE

- Positional:** Mapped strictly by relative order. `f(2, 3)` → `a=2, b=3`.
- Keyword:** Assigned explicitly by name. `f(b=3, a=2)` overrides order safely.



POSITIONAL FOLLOWS KEYWORD RULE

POSITIONAL ARGUMENTS MUST ALWAYS PRECEDE KEYWORD ARGUMENTS INSIDE ANY CALL STATEMENT!

`print("Hi", end="", "World")` → ❌ **SyntaxError: positional argument follows keyword argument**

5. STRING METHODS SURVIVAL BOX

METHOD SYNTAX	OUTPUT VALUE	EXAM TRAP / BEHAVIOR
<code>"hi".upper()</code>	<code>"HI"</code>	Strings are immutable; methods return a <i>new</i> string string. Original stays lowercase.
<code>"HI".lower()</code>	<code>"hi"</code>	Converts all casing characters to lowercase.
<code>"txt".replace("t","x")</code>	<code>"xxxl"</code>	Replaces all occurrences of the substring character.
<code>"banana".find("an")</code>	<code>1</code>	Returns lowest starting index . Returns <code>-1</code> if substring is missing!
<code>"a,b".split(",")</code>	<code>['a', 'b']</code>	Splits string into a list of strings at delimiter points.
<code>"-".join(['a', 'b'])</code>	<code>"a-b"</code>	Joins list elements into string using separator. Fails if items are not strings!

6. TRUTHY VS FALSY SECTOR

STRICTLY FALSY VALUES CHECKLIST:

- `False` • `None`
- `0` (int) • `0.0` (float)
- `""` (empty string) • `[]` (empty list)
- `{}` (empty dict) • `set()` (empty set)

💡 RULE: EVERYTHING ELSE IS TRUE!

Non-empty string literals evaluate to True regardless of text content value.

`bool("False")` → `True` • `bool("0")` → `True`

7. CORE SCOPES & PCEP TERMINOLOGY MATRIX

Global Scope: Variables defined outside functions. Visible everywhere inside file.

Local Scope: Variables defined inside a function block. Visible only within that function execution context.

Shadowing: Variable assigned inside function automatically becomes local, masking global variables. Global stays 10: `x=10; def f(): x=5`

Keyword: Reserved word token with a fixed syntactical purpose (`def`, `while`, etc.). Can never be variable identifiers!

1. LOOPS DEDICATED REVIEW

A. FOR LOOP (Loops through a sequence)

```
for x in range(3):  
    print(x)
```

TRANSLATION:
FOR EACH VALUE IN A SEQUENCE → DO SOMETHING.

B. WHILE LOOP (Repeats based on condition)

```
while x < 3:  
    x += 1
```

TRANSLATION:
REPEAT AS LONG AS CONDITION IS TRUE.

C. Loop Control Commands

```
break  
continue  
pass
```

🔥 INFINITE LOOP & UPDATE TRAPS

- Loops hang forever if conditional parameters never evaluate to False.
- **Exam Trap:**** Forgetting to update counter variables (`x += 1`) creates infinite runs.
- **Exam Trap:**** `continue` immediately skips the rest of the current loop layer code block.

2. RANGE() EXPLORATION TERMINAL

MANTRA: START INCLUDED — STOP EXCLUDED

```
range(stop)           → Starts at 0, steps 1.  
range(start, stop)    → Explicit start, steps 1.  
range(start, stop, step) → Moves by step factor.
```

SYNTAX EXAMPLE	GENERATED SEQUENCE VALUES
<code>range(5)</code>	<code>0, 1, 2, 3, 4</code>
<code>range(1, 5)</code>	<code>1, 2, 3, 4</code>
<code>range(2, 13, 3)</code>	<code>2, 5, 8, 11</code>
<code>range(1, 2)</code>	<code>1</code>
<code>range(5, 1, -1)</code>	<code>5, 4, 3, 2</code> (Counts down backward)

🔥 THE ABSOLUTE STOP VAL BOUNDARY RULE

THE STOP VALUE NEVER APPEARS IN THE OUTPUT SEQUENCE.

`range(1, 5)` will generate up to 4. It never yields 5. If increments are impossible (`range(1, 5, -1)`), it yields an empty loop without throwing errors.

3. DATA STRUCTURES COMPARISON MATRIX

TYPE	SYNTAX	ORDERED	MUTABLE	DUPLICATES
list	<code>[1, 2]</code>	YES	YES	YES
tuple	<code>(1, 2)</code>	YES	NO	YES
dict	<code>{"k":v}</code>	NO	YES	Keys: NO
set	<code>{1, 2}</code>	NO	YES	NO

Common List Manipulation Methods:

```
.append(x)  
.insert(i, x)  
.remove(x)  
.pop(i)
```

🔥 EMPTY STRUCTURES & SINGLE ITEM TRAPS

- `{}` creates an empty **DICTIONARY**, NOT a set!
- `set()` is required to instantiate an empty set.
- `(5)` evaluates as plain integer. Single item tuples require trailing commas: `(5,)`.

4. SEQUENCE SLICING & BUILT-IN HELPER CHECKLIST

Slicing Axiom: `seq[start:stop:step]` → Start included, stop excluded.
`"PCEP"[1:3]` yields `"CE"`. Indexing starts at 0.

Tolerance: Out-of-bounds slices clip cleanly instead of throwing error crashes: `[1, 2][0:99]` safely returns `[1, 2]`.

Core Helpers: `len()` returns element count. `type()` checks type classes. `print()` outputs strings but returns `None`!

User Input: `input()` always captures keyboard data streams as a `str` string, requiring manual numeric conversions.

1. PRECEDENCE LADDER & IDENTITY

1 (HIGH)	()	Parentheses override everything.
2	**	⚠️ Power runs Right-to-Left : $2*3**2 = 2**(3**2) = 512$.
3	+x, -x	Unary signs. Right of power wins: $2*-2 = 0.25$.
4	*, /, //, %	Multiplicative math. Evaluates Left-to-Right.
5	+, -	Additive math layers (Binary plus and minus).
6	<<, >>	Bitwise Logical Left/Right Shifts.
7	& → ^ →	Bitwise operations hierarchy ordering (AND → XOR → OR).
8	=, is, in	Equality value match, Identity match, Membership match.
9 (LOW)	not → and → or	Logical boolean layer evaluation stack order (NOT → AND → OR).

Practical Priority Examples:

3 << 2 | 1

2 + 3 * 4

$3 \times 4 = 12$

$3 \times 4 = 12$

$12 | 1$

14

⚠️ IDENTITY VS PARITY EQUALITY

• `=` checks if values are equal.

• `is` checks strict unique address identities.

• `in` / `not in` evaluate membership existence inside sequence iterables.

2. COMMON PCEP QUESTION DECODER

• list vs tuple

When you see...	What is the exam testing?
<code>def</code> / <code>return</code>	Function declaration parameters and immediately exiting value loops.
<code>range(x, y)</code>	Loop constraints boundary loops. Target <code>y</code> is excluded!
Mutability differences. Modifying tuple indexes throws errors.	
<code>is</code> vs <code>==</code>	Address location identity vs simple content value matching.
<code>try</code> / <code>except</code>	Catching runtime issues safely before scripts crash out.
<code><<</code> / <code>>></code>	Fast binary bit manipulation operations (multiplication/floor division).

3. BITWISE SURVIVAL MATRIX (4-BIT PILOT)

Dec	8	4	2	1	Binary	Dec	8	4	2	1	Binary
0	0	0	0	0	0b0000	8	1	0	0	0	0b1000
1	0	0	0	1	0b0001	9	1	0	0	1	0b1001
2	0	0	1	0	0b0010	10	1	0	1	0	0b1010
3	0	0	1	1	0b0011	11	1	0	1	1	0b1011
4	0	1	0	0	0b0100	12	1	1	0	0	0b1100
5	0	1	0	1	0b0101	13	1	1	0	1	0b1101
6	0	1	1	0	0b0110	14	1	1	1	0	0b1110
7	0	1	1	1	0b0111	15	1	1	1	1	0b1111

`&`

= COMMON BITS: 1 only if BOTH bits are 1. $10 \& 12 \rightarrow 8$.

`|`

= ALL BITS: 1 if at least one side is 1. $10 | 12 \rightarrow 14$.

`^`

= DIFFERENT BITS ONLY: 1 if bit states differ. $10 \wedge 12 \rightarrow 6$.

`<<`

Left Shift: Multiply left side by 2^{right} . $2 \ll 3 \rightarrow 2 \times 8 = 16$.

`>>`

Right Shift: Floor divide left side by 2^{right} . $11 \gg 2 \rightarrow 11 // 4 = 2$.

⚠️ UNARY OPERATOR ASTERISK REJECT

Multiplication symbols (`*`) are strictly binary. Running `print(*5)` triggers a fatal syntax block crash!

4. EXCEPTION ARCHITECTURE DECODER

```
try:
    # Code execution lines run path here.
except ArithmeticError:
    # Catches parent groups and subclass children!
except ZeroDivisionError:
    # ⚠️ This line is dead block if placed below parent!
finally:
    # Guaranteed line execution regardless of errors.
```

5. HIGH-FREQUENCY CORE 6 ERRORS

ERROR NAME	COMMON CAUSE / EXAMPLE
NameError	Using variable before definition. <code>print(undef)</code>
ValueError	Right type, wrong internal value context. <code>int("xyz")</code>
TypeError	Operation applied to incompatible types. <code>len(42)</code>
IndexError	Sequence array lookup index out of bounds. <code>[1][9]</code>
KeyError	Target key lookup missing from dictionary. <code>{ }["k"]</code>
ZeroDivisionError	Dividing evaluation by zero or zero floats. <code>1 / 0.0</code>

⚠️ THE ULTIMATE EXAM ROOM TRAPS CHECKLIST (REVIEW RIGHT BEFORE ENTERING!)

• **Identifier vs Value:** Changing label bindings via assignment does not mutate old data inside memory structures.

• **Parameter vs Argument:** Parameters act as labels inside definitions; arguments provide raw value inputs inside call blocks.

• **Print vs Return:** `print()` shows visual outputs but evaluates to `None`. Only `return` chains active object payloads out.

• **Finite range() Exclusions:** `range(5)` yields 0,1,2,3,4. The stop value 5 is never generated or accessed.

• **Float-String roadblocks:** Running `int(float("5.5"))` breaks with a ValueError. Change down sequentially: `int(float("5.5"))`.

• **Missing Tuple Comma:** Coding `x = (5)` instantiates an integer. Single items require trailing commas: `x = (5,)`.

• **Case Sensitivities:** Tokens like `true` or `none` are valid variable labels. Only `True`, `False`, and `None` are keywords.

• **Default Parameter Constraints:** Preset positional parameters (`def f(a, b=2):`) must sit strictly behind raw variable placeholders.

• **Bitwise vs Logical:** `and` / `or` short-circuit boolean valuations. `&` / `|` evaluate individual bits step-by-step.